
OPTIMIZING ACADEMIC TIMETABLING WITH PROXIMAL POLICY OPTIMIZATION: HUMAN JUDGMENT VS. ALGORITHMIC SCHEDULING

Mimjamam Ul Haque Monmoy^{1†}, MD. Hasib Sarkar^{1†}, Kaniz Faria Ahamed¹,
Airin Akhter¹, Dr. Debajyoti Karmaker^{1*}

¹Department of Computer Science and Engineering,
American International University - Bangladesh, Dhaka 1229, Bangladesh

ABSTRACT

University faculty course assignment is a difficult task, determined by the likes of preference, equitable distribution of workloads, schedule compactness, and adherences to institutional policies. While heuristic and metaheuristic approaches have been common, these frequently fail to individualize allocations or adapt to variable academic demands. The present work reformulates the University Course Assignment Problem (UCAP) as a constrained Markov Decision Process and applies a reinforcement learning approach under the Proximal Policy Optimization algorithm in a self-built CourseAssignmentEnv. In the constructed framework, the multi-dimensional reward scheme serves the dual purpose of enforcing hard constraints like prevention of timetable conflicts as well as respect for faculty eligibilities simultaneously with softer goals such as satisfaction and work-life balance.

The model was stringently tested on actual university data, with comprehensive records both for faculty and course offerings across one full term. Compared with human-crafted schedules as well as with an interpretable rule based approach, the RL method showed outstanding ability to meet instructor preferences such that, in every case, 100% preference match was achieved plus all conflicts avoided bettering the heuristic (99.36%) and human (77.29%) controls in this respect. However, the RL method did not fully book all the courses, with a total number of sections unassigned as 118 (compared with 28 for the heuristic as well as zero for human scheduling). Moreover, although the RL agent was resilient towards constraints, it was disappointing with respect to loading workload as well as compactness optimization, areas in which human expertise is still better, as evidenced by the Gini coefficient as well as by the variance in the distribution of the workload. These results represent the hope and the limitation today of deep reinforcement learning in course schedule construction. This work provides a new baseline for the use of RL in this domain and indicates the imperatives for future work on interpretability, scalability, and deeper embedding with institutional constraints towards practicality in the real world.

Keywords University Course Scheduling · Faculty Assignment · Deep Reinforcement Learning · Proximal Policy Optimization · Heuristic Algorithms · Timetabling · Workload Fairness · Preference Satisfaction · Constraint Optimization · Academic Resource Allocation

1 Introduction

The university course assignment problem is one of the cornerstones in education planning that directly affects the quality of education, faculty morale, and the general efficiency of the institution. UCAP, the process, is more than just a logistical assignment; it is a collection of multiple, interrelated constraints and goals that need simultaneous satisfaction. Administrators must take into account faculty schedules, personal preferences, departmental equitable workloads, room capacities, the reduction of conflicts, all the while meeting the policy constraints and the necessities of the curriculum. Being, by definition, a problem whose formulation is highly combinatorial, high dimensional, and

*Corresponding author: debajyoti.karmaker@auiub.edu

frequently time varying in the domain that defines the constraints, UCAP is definitely an NP-hard problem in operational research [1, 2]. In the past, universities have approached course assignment as solving a UCAP via metaheuristic approaches like Simulated Annealing, Tabu Search, and Genetic Algorithm. Those methods have performed well under the test cases, given that these approaches have flexibility in the way the constraints are embedded, as well as the capability to discover feasible regions in the vast, complicated exploration spaces [2, 1]. However, they often require labor intensive parameter tuning and are not easily adaptable, as they cannot meet the dynamic, real-world needs of the educational institutions. In addition, they often disappoint when attempting to prioritize softer goals like educator delight and the eventual, prolonged fairness nor can they easily recover when faced with last minute changes, such as the unavailability of faculty or changes in demand that occur at the last minute [3, 4]. More recently, machine learning approaches have found their way into the field, as the authors have employed supervised learning models that forecasted timetable feasibility and faculty allocation, all according to expressed preference [3, 4]. Some have developed hybrid approaches that integrated rule-based heuristics and predictive modeling, all in the name of adaptability [5, 6]. While these have shown promise, as measured by certain metrics like conflict reduction, or enhanced satisfaction level there is still the limitation that the sets of the rule are static, as is the way the data sets that are consulted, meaning these methods cannot generally be taken across a variety of the educational contexts, nor across the scale that the institutions can encompass. Reinforcement learning, and particularly deep reinforcement learning, has brought new hope into the possibilities that can be brought toward these complex, complicated scheduling problems. Differently from rule based or purely supervised methods, RL agents learn optimal assignment policies through interacting directly with a simulated environment, learning through the receiver reward or penalties as they try out various course faculty assignments. Most recent studies have indicated that RL agents can utilize policy networks, attention, and graph based data representations in order to learn the subtleties of the schedule constraints, allowing them to trade off multiple conflicting objectives as well as generalize across diverse problem instances [7, 8, 9, 10, 11, 12]. Most importantly, DRL models have achieved promising outcomes in neighboring areas such as the job-shop schedule, production schedule, and resource allocation [7, 8, 9]. In spite of these recent developments, the application of RL into university schedules has special opportunities as well as challenges. It is a challenging task to design useful reward structures that accurately encode hard institutional constraints (e.g., conflict evasion, eligibility) as well as soft goals (e.g., schedule fairness, instructor satisfaction) [13, 14, 15, 16]. Others include dealing with sparse or delayed rewards, overfitting to certain schedule patterns, as well as ensuring that the derived policies are human-interpretable [16]. Besides these, robust integration with data systems found in institutions and scalability across the semester as well as across departments remain key challenges towards the real-world applicability [17, 18]. In attempting the gaps, this work formulates the University Course Assignment Problem as a constrained version of the Markov Decision Process as well as develops a special-purpose RL environment *CourseAssignmentEnv*—that accurately models the intricate schedule constraint interchance schedule goals seen in actual institutions. It is trained through the Proximal Policy Optimization (PPO) algorithm, informed through a specially crafted multi-objective reward function that is designed carefully. It is through extensive experiments on actual world datasets that capture detailed faculty facts as well as detailed course information across the full semester that the authors hope to capture the depth as well as the complexity seen across actual institutions. It is through the work that the authors wish not just to indicate the viability of deep reinforcement learning in the automated as well as the enhancement of the schedule, but rather elucidate the trade offs as well as the challenges that exist towards the efficiency as well as the balancing across the schedule, the schedule, as well as the schedule feasibility. The results provide new insights into the abilities and the current constraints of RL-based timetabling, which can be the basis for further work aimed at interpretability, scalability, and the adoption of more developed AI approaches into university decision making practices.

2 Literature Review

The University Course Assignment Problem (UCAP) is a specific scheduling problem dealing with the assignment of university courses to suitable instructors, rooms, and timeslots, subject to many institutional constraints. Institutional constraints are common and often include instructor availability, room capacities, curriculum cohesiveness, departmental balancing of workloads, and course conflict minimization. Because UCAP is combinatorial and heavily institutional, UCAP is also considered an NP-hard problem, often too hard as a hard problem, and thus often solved with heuristic or machine learning-based approaches.

Classical methods of course assignment have largely employed metaheuristics like Simulated Annealing, Tabu Search, and Genetic Algorithms. Burke et al. [19] introduced a hybrid genetic algorithm specific to exam and course timetabling, which employed constraint repair mechanisms to enhance feasibility levels. Lewis [1] conducted a general survey on such metaheuristic approaches, including common hard and soft constraints in university environments. They saw acceptable performances on benchmarker sets, though they needed heavy manual parameter setting and had no response to dynamic scheduling cases.

In response, machine learning (ML) methods have been tested to augment or substitute rule-based heuristics. Kenekayoro [3] formulated a supervised classifier to test timetable feasibility, reaching a 96% accuracy level and holding the potential for real-time response in iterative assignment frameworks. Umamaheswari et al. [20] went further by introducing a faculty allocation framework through supervised learning that adjusted when answering any given instructor's schedule and shifting workloads. Their framework lowered conflicts in the schedule by 21% compared to customary assignment policies.

Application of reinforcement learning (RL) and, in particular, Deep Reinforcement Learning (DRL) has brought a new paradigm shift towards adaptive and scalable education schedules. Liu et al. [7] put forward the Actor-Critic DRL framework for the job-shop schedule, whose makespan was lowered by 7–9% in respect to rule-based dispatching. Despite being not aimed towards course assignment, the incorporation of policy and value networks was providing foundational insights towards later UCAP implementations. Bonetta et al. [8] put forward a sequence-to-sequence DRL framework towards scheduling, whose attention-based encoder mapped the job instances onto dispatching sequences, providing 10–12% better performance compared to baseline RL agents.

Haider et al. [5] proposed a hybrid DRL-genetic formulation for scheduled courses in intelligent education systems. Their two-stage framework initially came up with allocations through DRL and later tuned them through the use of evolutionary operators, achieving 93.1% hard constraint satisfaction and 84.5% soft constraint optimization. Ghaffar et al. [6] showed that hybridization through DRL with swarm intelligence, like Ant Colony Optimization, resulted in a feasibility enhancement of 16.7% over 22 curriculum-aware datasets.

Personalization and student-adaptive allocation have also been considered. Bassen et al. [21] employed DRL to develop personalized learning content paths, boosting the level of engagement among students by 12% compared to stationary schedules. Xu et al. [22] developed an RL agent that was tuned towards the optimal, student-specific sequence of courses given the graduation requirements, without comprehensive timetable conflict resolution.

Reward shaping has become a critical part in DRL for limited environments such as UCAP. Reward designs have been introduced by Yu and Zhao [13] for sparse-feedback environments, which helped converge 35% faster in their experiments. Weight anchoring improved training robustness by Gracla et al. [14], which improved policy reliability in noisier or uncertain constraints by 22%.

Graph Neural Networks (GNNs) have further improved DRL performance through structured relationship modeling in scheduling environments. Song et al. [9] proposed a DRL+GNN scheduler for flexible job-shop environments, reducing the makespan by 12% through the use of job-machine dependency graphs. Park et al. [10] employed joint GNN-policy learning, promoting generalization toward previously unseen scheduling conditions. Hameed and Schwung [11] transferred analogous concepts toward production schedule modeling, as their GNN-based policies increased robustness under varying load conditions by 15%.

Multi-agent reinforcement learning (MARL) is another promising avenue. Pu et al. [23] developed a decentralized scheduler with cooperating agents that reached 8–11% increased throughput under dynamic workloads. For UCAP, the latter architecture supports modeling departmental or instructor-specific agents, but the problem is open for coordinating them, as well as scalability.

Paduraru et al. [24] developed a DQN-based constrained task allocator that reached 90% synthetic course-like environments' constraint satisfaction. Jain and Jyoti [25] developed HDRLGA, a DRL-genetic hybrid, which exceeded cloud scheduling baselines by 14% in average task accomplishment, showing high suitability towards adapting such models to academic course allocation. Demirel et al. [26] introduced DEEPCAS, a control-aware DRL framework with LSTM elements, illustrating how temporal dependencies can be represented in academic environments with multi-week schedules.

In the base work, Goh et al. [27] improved simulated annealing through adaptive reheating when applied to educational scheduling, achieving up to a 10% increase in soft constraint satisfaction. Nguyen et al. [28] introduced system-level DRL frameworks at the suggestion level as proposed for the construction of.

Fan et al. [18] utilized DRL on dynamic high-performance computing task scheduling with transferable experience on load-aware course allocation systems. Guo [15] investigated reward shaping and time-varying state encoding on DRL-based intercity railway scheduling, approaches that can be directly applied on multi-period UCAP models. Lastly, Khadivi et al. [16] performed a meta-review on DRL in machine scheduling, and reward sparsity, overfitting by policy, and handling constraints were found as key challenges. Their advice was the integration of domain knowledge along with the use of curriculum learning towards improved convergence—keywords particularly useful in constrained, policy-based university schedule systems.

In short, UCAP research evolution is one from the rule-based, static approaches towards learning-based adaptive frameworks. DRL is particularly useful in achieving flexibility, satisfiability of constraints, and generalization. Inte-

gration among GNNs, reward engineering, and hybrid metaheuristics is still expanding the front. However, practical use still requires increased interpretability, integration with rule-based institutions, and scale-friendly handling of personalization.

3 Methodology

The university course scheduling problem is modeled as a constrained combinatorial optimization problem. The objective is to allocate a set of university courses to eligible faculty members in a manner that satisfies institutional constraints, honors faculty preferences, balances workloads, and avoids scheduling conflicts.

Let:

- $C = \{c_1, c_2, \dots, c_n\}$ be the set of n courses offered in a semester.
- $F = \{f_1, f_2, \dots, f_m\}$ be the set of m faculty members available for teaching.

Each course c_i is defined by a tuple:

$$c_i = (\text{class_id}, \text{course_title}, \text{type}, \text{section}, \text{schedules}, \text{duration})$$

Where:

- $\text{type} \in \{\text{Theory}, \text{Lab}\}$
- schedules is a list of tuples $(\text{day}, \text{start_time}, \text{end_time})$
- duration is the total number of teaching minutes

Each faculty f_j is defined by a tuple:

$$f_j = (\text{faculty_id}, \text{designation}, \text{max_courses}, \text{course_type}, \text{preferences})$$

Where:

- designation can affect prioritization or course eligibility
- max_courses is the upper bound on how many courses a faculty can be assigned
- $\text{course_type} \in \{\text{Theory}, \text{Lab}, \text{Mix}\}$
- preferences is a ranked list (up to 5) of preferred course titles

3.1 Objective

Determine a valid assignment set $A \subseteq F \times C$, such that:

- Each course $c_i \in C$ is assigned to at most one faculty $f_j \in F$.
- The assignment maximizes a composite reward function :
 - Faculty preference satisfaction
 - Load balancing (teaching hours and course counts)
 - Time gap minimization (compact schedule)
 - Course type diversity
 - Long-term workload fairness (approximated using future projection heuristic)

3.2 Hard Constraints

The following conditions must be met for a course c_i to be assigned to a faculty f_j :

- **Uniqueness:** Each course is assigned only once.

$$\forall c_i \in C, \quad \exists \text{ at most one } f_j \text{ such that } (f_j, c_i) \in A$$

- **Preference Inclusion:** The course c_i 's title must appear in f_j 's preferences.
- **Time Conflict Avoidance:**

No overlap between c_i 's schedule and existing assigned courses of f_j

- **Teaching Hour Limit:**

$$\text{Total minutes assigned to } f_j \leq 21 \times 60$$

- **Course Count Limit:**

$$\text{Number of courses assigned to } f_j \leq f_j.\text{max_courses}$$

- **Course Type Eligibility:**

$$c_i.\text{type} \in \{\text{Mix}, f_j.\text{course_type}\}$$

3.3 Soft Objectives (Reward Maximization)

Each feasible assignment (f_j, c_i) is evaluated using a multi-objective reward function:

$$R(f_j, c_i) = \alpha_1 \cdot \text{Preference Score} + \alpha_2 \cdot \text{Load Balance Score} + \alpha_3 \cdot \text{Schedule Compactness} + \alpha_4 \cdot \text{Type Diversity Bonus} + \alpha_5 \cdot \text{Future Fairness}$$

Where each α is a tuned hyperparameter. The reward helps the reinforcement learning agent or heuristic select optimal actions during assignment.

A. Faculty Dataset

This dataset contains one entry per faculty member, encapsulating both administrative constraints and individual teaching preferences. The key attributes are:

Faculty ID:	A unique identifier for each instructor.
Faculty Name:	Full name of the faculty member.
Designation:	Academic rank (e.g., Lecturer, Assistant Professor, Associate Professor, Professor, Instructor). This is useful for analyzing capacity and eligibility patterns.
Max Courses:	The maximum number of courses a faculty member is allowed to teach during the semester. This constraint reflects institutional policy and varies by designation.
Course Type Eligibility:	Indicates whether the faculty can teach Theory, Lab, or both (Mix). This attribute directly impacts assignment feasibility.
Preferences:	A ranked list (up to 5) of preferred course titles. The list reflects faculty teaching interest or prior experience. Preferences are critical for computing preference-based reward or scoring during assignment.

B. Course Dataset

This dataset includes detailed information about all courses offered in the given semester. Each course may include multiple sections, schedules, and metadata necessary for constraint enforcement. Key attributes include:

Class ID:	Unique identifier for the course section.
Course Title:	Descriptive name of the course (e.g., “Artificial Intelligence”).
Section:	Alphanumeric code identifying the specific offering (e.g., A1, B2).
Type:	Either Theory or Lab. Impacts the eligibility match with faculty.
Total Duration:	Total number of teaching minutes required (aggregated over all weeks).
Day(s):	The weekday(s) the course is scheduled (e.g., Monday, Wednesday).
Start Time / End Time:	Time window of the scheduled session(s). A course may span multiple non-consecutive sessions during the week.

These datasets are loaded and preprocessed before being passed to either the heuristic or RL environment. During environment resets, both the faculty and course states are re-initialized to simulate realistic assignment scenarios.

3.4 Custom Reinforcement Learning Environment

In order to overcome the intricate and constraint-based property of university course scheduling, we have created a special-purpose OpenAI Gym environment, which we call `CourseAssignmentEnv`. It simulates the scheduling problem as the Markov Decision Process (MDP) that facilitates the use of reinforcement learning to apply sequential course assignment on faculty members with real-world constraints.

3.4.1 Agent-Environment Interaction

The agent interacts with the environment by proposing course-to-faculty assignments based on its policy. Each action is evaluated using a custom reward function and the environment updates its internal state accordingly. This interaction loop is illustrated in Figure 1:

- **Policy (MLP):** The agent uses a multi-layer perceptron (MLP) to map the current state of the environment to an action — specifically, selecting a course and assigning it to a faculty member.

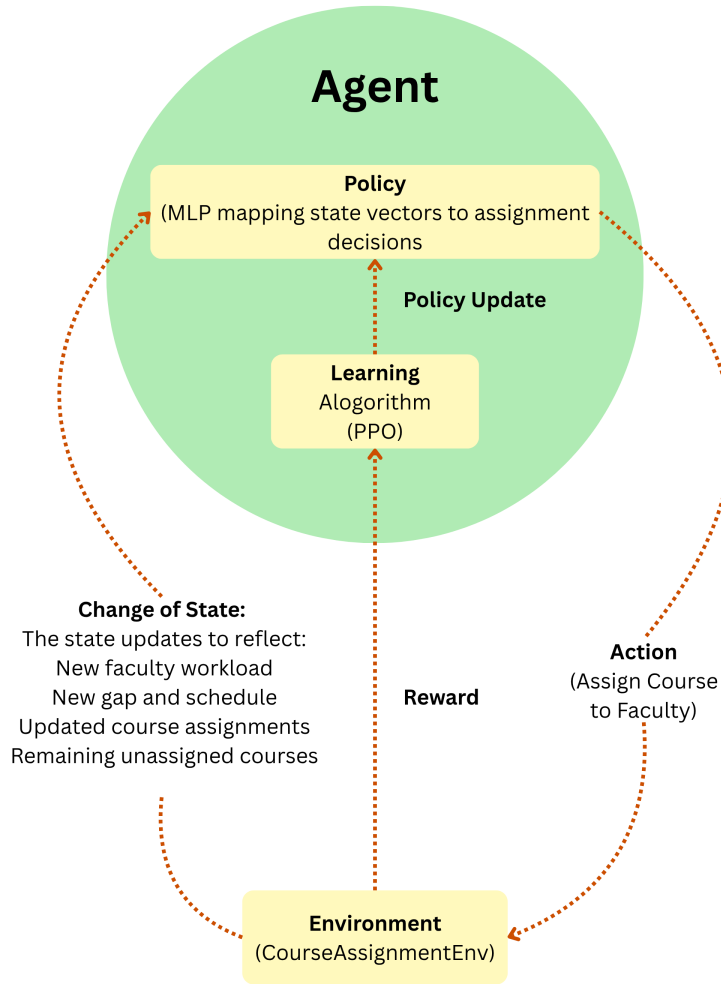


Figure 1: Agent-Environment Loop for Course Assignment

- **Action:** The selected action is executed in the environment, representing a course assignment.
- **Environment Update:** The environment updates internal representations, including workload, schedules, and assignments, and returns a reward and new state.
- **Learning Algorithm (PPO):** Proximal Policy Optimization (PPO) uses the reward feedback to adjust the policy through gradient-based updates.

This closed-loop system enables the agent to iteratively improve its course assignment policy based on observed outcomes.

3.4.2 Observation Space (State Vector Representation)

The state vector is a critical element in the learning process, capturing all relevant features needed by the policy to make informed decisions. Each faculty member is represented by a vector that encodes both hard constraints and dynamic conditions. The complete observation space is a matrix of shape $(\text{num_faculties}, \text{num_courses} \times 2 + 4)$.

The state vector for each faculty includes:

- **Base Features (4 values):**
 - **Normalized Workload:** Current teaching load divided by the maximum allowed (21 hours/week).
 - **Remaining Course Capacity:** Ratio of courses that can still be assigned.

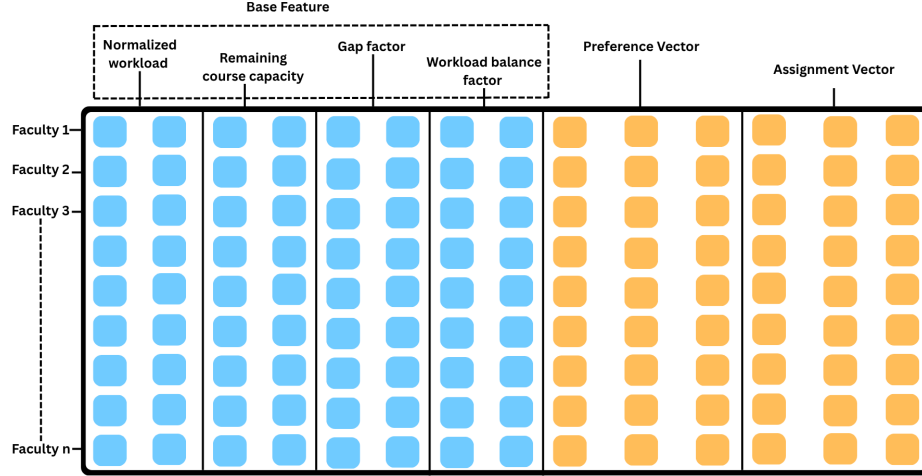


Figure 2: State Vector Structure

- **Gap Factor:** Average time gaps between scheduled sessions, normalized.
- **Workload Balance Factor:** Measures deviation from median workload across faculty.
- **Preference Vector (num_courses):**
 - Encodes ranked preferences of the faculty member for each course.
 - Scaled from 1.0 (top preference) to 0.0 (not preferred).
- **Assignment Vector (num_courses):**
 - A binary vector indicating which courses are already assigned to the faculty.

Each row in the matrix corresponds to a faculty member and the full matrix represents the current state of all faculty in the system.

3.4.3 Action Space

The action space is defined as a tuple (faculty_index, course_index). This discrete representation allows the agent to select a valid pair for assignment during each interaction step. Only one assignment is made per step, and the environment automatically rejects invalid actions (e.g., duplicate assignments or violating constraints) via negative rewards.

3.5 Reward Components Used in CourseAssignmentEnv

Table 1: Reward Components used in Course Scheduling

Reward Component	Description	Weight / Behavior
Preference Match	Based on the rank of preferred course	+5 to +25
Load Balance	Inverse standard deviation of total assigned hours	+ up to 12
Schedule Compactness	Fewer gaps between classes	+ up to 6
Course Type Match	Matching “Theory”, “Lab” or “Mix”	+1 or 0
Type Diversity	Higher reward for variety	+ up to 8
Over-assignment Penalty	For exceeding max course count	-5 per overload
Time Conflict Penalty	If assigned course overlaps with another	-15
Long-term Fairness Bonus	Simulated workload variation among faculty	up to +5

Reward Function Design

To guide learning, the reward function integrates multiple components:

- **Preference Satisfaction:** Higher reward for assigning courses that are top-ranked in faculty preferences.
- **Workload Balance:** Penalizes imbalance in total teaching load across faculty.
- **Gap Minimization:** Rewards compact schedules with minimal time gaps.

- **Course Type Diversity:** Encourages assignment of varied types (e.g., lab and theory).
- **Assignment Completeness:** Positive reward for progressing toward full course allocation.
- **Constraint Adherence:** Negative rewards for conflicts, overloads, or ineligible assignments.

Reward Function Engineering

The weights for each component were determined through empirical tuning and domain expertise rather than automated optimization. Initial values were selected according to the relative institutional importance of each scheduling objective, for example, higher emphasis was placed on preference satisfaction and conflict avoidance, as these directly affect feasibility. Subsequent refinement followed an iterative trial-and-error process, where the PPO agent was trained under different weight configurations for short sessions, and its performance was evaluated using metrics such as constraint violations, reward stability, and workload fairness. The most balanced configuration was then chosen based on stable convergence and practical interpretability of the resulting schedules.

During early experimentation, it was observed that certain components, particularly Preference Satisfaction and Load Balance could interfere with one another if their magnitudes were not properly scaled. For instance, overemphasizing preference satisfaction often led to highly unbalanced workloads, while excessive weighting on load balance reduced the number of feasible assignments. To mitigate such effects, all reward values were normalized and clipped within the range $[-30, 30]$, ensuring smoother PPO updates and preventing unstable gradient behavior.

Clarification of the Long-term Fairness Bonus: The "Long-term Fairness Bonus" was implemented as a heuristic to approximate the future impact of a current assignment on overall workload distribution. Specifically, it estimates the potential future workload variance among faculty if the current assignment is made. It is calculated by projecting the new workload state after the assignment and computing the inverse of the standard deviation of the projected workloads across all faculty. A higher reward (up to +5) is given for assignments that lead to a more equitable projected distribution. While this is a simplified projection and does not involve a full future trajectory simulation, it provided a crucial signal to nudge the agent away from myopic decisions that would lead to severe long-term imbalance.

3.6 Explanation of PPO Training Configuration

To train our reinforcement learning agent for course-faculty assignment, we employed the Proximal Policy Optimization (PPO) algorithm using the Stable-Baselines3 library. The hyperparameter settings, outlined in Table ??, were carefully selected to balance stability, convergence speed, and exploration. Here's a breakdown of each parameter:

Table 2: Proximal Policy Optimization (PPO) Training Configuration

Parameter	Value
Total Timesteps	500,000
Learning Rate	3×10^{-4}
Batch Size	1024
GAE Steps (n_steps)	2048
Discount Factor (γ)	0.99
Clip Range	0.2
Entropy Coefficient	0.2
Policy	MLP Policy (2-layer MLP)

- **Total Timesteps:** The agent was trained for 500,000 interactions with the environment, allowing sufficient exploration of assignment strategies and reward optimization.
- **Learning Rate (3×10^{-4}):** A moderately low learning rate was chosen to ensure stable convergence and avoid oscillations during policy updates.
- **Batch Size (1024):** Each PPO update used mini-batches of size 1024, allowing for efficient gradient computation without excessive memory overhead.
- **GAE Steps ($n_steps = 2048$):** This defines the number of steps to run for each environment before an update. Larger values provide more stable value estimates via Generalized Advantage Estimation (GAE).
- **Discount Factor ($\gamma = 0.99$):** The discount factor ensures the agent values future rewards almost as much as immediate ones, encouraging long-term load balancing and efficient scheduling.
- **Clip Range (0.2):** PPO uses a clipped objective to avoid large policy updates. A 0.2 clip range limits the deviation of new policies from old ones, maintaining learning stability.
- **Entropy Coefficient (0.2):** A relatively high entropy coefficient encourages exploration, promoting diverse assignment strategies during training.

- **Policy:** We used an MLP-based policy network (Multilayer Perceptron) with two hidden layers. It maps state vectors (representing faculty assignment states) to action probabilities over course-faculty pairs.

These settings were fine-tuned experimentally to accommodate the complexity of the scheduling problem, such as the large action space, hard constraints, and sparsity of high-reward scenarios.

This environment allows reinforcement learning agents to interact with a realistically constrained and richly structured scheduling system, enabling effective policy learning that generalizes across semesters and varying faculty-course configurations. The layered state space and carefully engineered reward signals ensure that the agent receives both local and global feedback for its actions, aligning with institutional goals of fairness, efficiency, and faculty satisfaction.

3.7 Mathematical Model of the Custom Environment

The course scheduling problem is formulated as a Markov Decision Process (MDP), where the agent aims to learn an optimal policy for assigning university courses to faculty members under hard constraints and soft objectives. The MDP is defined by the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$, where:

- $\mathcal{S}$ is the state space
- $\mathcal{A}$ is the action space
- $\mathcal{P}$ is the transition function
- $\mathcal{R}$ is the reward function
- $\gamma \in [0, 1]$ is the discount factor

3.7.1 State Space $\mathcal{S}$

The state $s_t \in \mathcal{S}$ at time step t is a matrix of shape $(N, 2M + 4)$, where:

- N : Number of faculty members
- M : Number of courses

Each row represents a faculty member and includes:

- 4 base features:
 - w_j : Normalized workload
 - c_j : Remaining course capacity
 - g_j : Normalized average gap
 - b_j : Workload balance factor
- M -dimensional preference vector $\vec{p}_j$, where $p_{j,i} \in [0, 1]$ denotes the normalized preference for course c_i
- M -dimensional assignment vector $\vec{a}_j$, where $a_{j,i} \in \{0, 1\}$ indicates if course c_i is already assigned to faculty f_j

3.7.2 Action Space $\mathcal{A}$

Each action $a_t \in \mathcal{A}$ is a discrete tuple (f_j, c_i) representing the assignment of course c_i to faculty member f_j . The full action space has size $N \times M$.

3.7.3 Transition Function $\mathcal{P}$

Given a state s_t and action a_t , the environment deterministically transitions to a new state s_{t+1} by:

- Updating the workload, schedule, and course list of faculty f_j
- Marking course c_i as assigned
- Recomputing all state features including gaps and balance

3.7.4 Reward Function $\mathcal{R}(s_t, a_t)$

The reward $r_t = \mathcal{R}(s_t, a_t)$ is a scalar value computed as:

$$r_t = \alpha_1 \cdot \text{PrefScore} + \alpha_2 \cdot \text{LoadBalance} + \alpha_3 \cdot \text{GapEfficiency} + \alpha_4 \cdot \text{TypeDiversity} + \alpha_5 \cdot \text{LongTermBenefit} + \alpha_6 \cdot \text{DupPenalty}$$

Where:

- **PrefScore:** Score based on how highly the faculty ranked the assigned course
- **LoadBalance:** Reward based on deviation from mean faculty workload
- **GapEfficiency:** Penalizes larger time gaps between classes

- **TypeDiversity:** Rewards diverse assignments across course types
- **LongTermBenefit:** Simulated impact of current action on overall fairness
- **DupPenalty:** Penalizes repeated assignments of the same course

The final reward is clipped to a range:

$$r_t \in [-30, 30]$$

3.7.5 Policy and Learning

The agent's policy $\pi(a_t|s_t; \theta)$ is parameterized by a neural network with weights θ . The PPO algorithm optimizes this policy by maximizing the expected discounted return:

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right]$$

with constraints on the magnitude of policy updates via clipped surrogate loss.

3.8 Weighted Heuristic Assignment Strategy

To establish a strong and interpretable baseline for comparison against the reinforcement learning approach, we designed a weighted heuristic assignment algorithm. This method mimics a rule-based scheduling system that evaluates faculty-course suitability using domain-informed scoring rules and hard constraints.

3.8.1 Overview of the Algorithm

The heuristic process follows a deterministic greedy approach. For each course in the list of offered courses, it evaluates all faculty members to identify the best match based on constraints and a weighted scoring function. The steps are visualized in Figure 3 (see flowchart).

3.8.2 Step-by-Step Flow

- Step 1: Start the Heuristic Assignment Process** Initialize the scheduling logic and prepare data structures for assignment tracking.
- Step 2: Load Faculty and Course Datasets** Import structured information about faculty preferences, maximum teaching loads, course types, and available courses for the semester.
- Step 3: Iterate Over Courses** Loop through each unassigned course to attempt a suitable faculty assignment.
- Step 4: Extract Course Details** Parse relevant features like course title, duration, timing, and course type (theory/lab).
- Step 5: Initialize Assignment Variables** For each course, reset `best_score` = $-\infty$ and `best_faculty_id` = None.
- Step 6: Iterate Over Faculty Members** Loop through all faculty members to evaluate their eligibility and fitness for the current course.
- Step 7: Check Eligibility Constraints** Faculty are only considered if they satisfy the following:
- Course appears in their preference list
 - Assigned workload + course duration ≤ 21 hours/week
 - Maximum course count is not yet reached
 - Course type compatibility (Theory/Lab/Mix)
 - No schedule overlap (time conflict check)
- Step 8: Compute Weighted Score** For eligible faculty, compute a weighted score to reflect assignment suitability. The scoring function is defined as:

$$\text{Score}(f, c) = 5p + 3r + 2l + 1.5g + 1t$$

Where:

- p : Preference score (higher if the course is ranked highly)
- r : Remaining course capacity $\left(\frac{\text{max} - \text{assigned}}{\text{max}} \right)$
- l : Inverse workload $\left(1 - \frac{\text{assigned minutes}}{1260} \right)$
- g : Gap efficiency (1 if no gap or minimized, 0 if inefficient)
- t : Type match indicator (1 if matched, 0 otherwise)

- Step 9: Score Comparison and Selection** Update `best_score` and `best_faculty_id` if the current faculty yields a higher score.



Figure 3: Heuristic Assignment Flowchart

Step 10: Assign Course to Best Faculty Once the loop completes, assign the course to the faculty member with the highest score and update their schedule and workload state.

Step 11: Repeat for All Courses Continue iterating until all courses are either assigned or no feasible faculty remains.

3.8.3 Mathematical Intuition

The heuristic scoring function is a linear combination of normalized assignment features, formulated as:

$$\max_{f \in F} \sum_{i=1}^5 \alpha_i \cdot x_i(f, c)$$

Where:

- $x_i(f, c)$ are the feature values (preference score, capacity, etc.)
- α_i are empirically chosen weights based on domain expertise

This formulation treats the assignment as a constrained maximization problem over a bounded, discrete action space. Although it lacks foresight or global optimization, it performs well in practical scenarios where rules are interpretable and constraints are strict.

3.9 Comparative Evaluation Framework

To assess the effectiveness of the proposed reinforcement learning (RL)-based scheduling agent, we compare its performance against two baselines:

- **Human-Generated Schedule:** The manually curated schedule created by academic coordinators for the same semester. This serves as the practical benchmark for feasibility and quality.

- **Heuristic Assignment Algorithm:** A deterministic rule-based approach that assigns courses based on a weighted scoring function (see Section 3.4). It reflects a fast and interpretable method without learning or adaptation.

Each of the three approaches operates on the same faculty and course datasets and is evaluated under identical constraints. We use the following common metrics for comparison:

- **Faculty Preference Satisfaction:** Average rank of assigned courses in each faculty’s preference list.
- **Workload Distribution:** Variance and standard deviation of total assigned teaching minutes among faculty.
- **Schedule Compactness:** Average daily time gap in a faculty member’s timetable.
- **Assignment Completeness:** Number of unassigned courses due to conflicts or constraint violations.
- **Conflict Count:** Number of detected time overlaps in each schedule.
- **Execution Time:** Time required to produce the schedule (measured for heuristic and RL models only).

This comparison enables a fair and quantitative assessment of the RL agent’s generalization capabilities and adaptability, while also validating the effectiveness of learning-based solutions over traditional human or heuristic strategies.

4 Results

The following section compares in detail three course scheduling approaches: manual (human) scheduling, weighted heuristic algorithm, and the RL agent trained using the PPO algorithm. All models were tested on identical faculty-course datasets under the same institutional constraints to ensure fairness and reproducibility. To give an immediate overview of the comparative performance, the key metrics for all three methods are consolidated in Table 3, with detailed faculty-level results given in Appendix A for reference.

Table 3: Summary of Key Scheduling Metrics Across Methods

Metric	Human (Manual)	RL (PPO Agent)	Heuristic
Preference Satisfaction (%)	77.29	100.00	99.36
Unassigned Courses (Count)	0	118	28
Workload Std. Dev. (hrs)	4.55	5.43	6.36
Gini Coefficient (Workload Inequality)	0.155	0.211	0.232
Average Schedule Gap (minutes)	88.5	95.9	85.8
Time Conflicts (Count)	0	0	49
Average Idle Time (hrs)	14.28	48.36	41.26

Note: Faculty-level workload and idle-time details are provided in Appendix A (Tables A1–A6).

4.1 Experimental Setup

An intensive, standardized experimental design was used such that comparative outcomes would be maintained in their integrity and be reproducible. Both the Reinforcement Learning and heuristic-based strategies for scheduling were comprehensively benchmarked through common, comprehensive datasets that captured actual institutional constraints as well as faculty-course attributes. Both paradigms for scheduling were given the challenge of assigning the full collection of semester-duration courses into the pool of faculty that is given, under strict constraints that are hard institutional constraints. Key optimization goals were as follows: (i) equitable distribution of work across faculty, (ii) as much as possible, meeting satisfaction of instructor preferences, and (iii) making feasible and realistic the schedules that resulted from applying the paradigms.

The RL approach was based on a PPO agent operating within a customdesigned Gym environment: `CourseAssignmentEnv`. This environment modeled the course assignment process as one involving sequential decision making, wherein an agent is iteratively picking a faculty-course pair. The reward function contained multifaceted criteria: preference satisfaction, schedule compactness, and balanced workloads, while infeasibility and constraints violations contributed to penalties. Training ran for 100,000 timesteps with extensive instrumentation and monitoring supporting performance inspection.

In parallel, the heuristic-based strategy employed a sophisticated, rule-driven scoring function for course assignment, developed from weighted criteria including faculty preferences, workload capacity, and compatibility constraints, and intra-day gap minimization, with strict adherence to all institutional hard constraints. Identical input datasets—a detailed profile of both faculty and courses—were used by both approaches, thus ensuring a correct and direct comparison. The outputs of all methods were subjected to quantitative and qualitative analysis, including direct comparison with human-generated (manual) schedules produced by experienced administrators.

4.2 Quantitative Comparison

This section summarizes key findings from quantitative measures of workload fairness, schedule compactness, conflict avoidance and course assignment completeness. Here, aggregate results across all faculty are summarized; details at the faculty level are given in Appendix A.

4.2.1 Workload and Idle Time Distribution

Workload distribution still is a key indicator of equity within course scheduling. The manual scheduling approach yielded the most even distribution, wherein the workload standard deviation was 4.55 hours and the average idle time was 14.28 hours. Though the RL agent had a much better optimization of preferences, its workload disparity was higher (standard deviation 5.43 hours) with considerable increases in idle times (48.36 hours), indicating an uneven distribution of workload among faculty. Meanwhile, the heuristic model presented the greatest dispersion (standard deviation 6.36 hours) with an average of 41.26 hours in idle time, reflecting over-optimization for preference and compactness at the expense of equity. *Detailed faculty-level workload and idle-time distributions for each method are provided in Appendix A (Tables A1–A6).*

4.2.2 Visualization and Comparative Metrics

Workload Fairness The standard deviation of the faculty workload is a direct measure of fairness. It can be seen from Figure 4 that manual scheduling has the lowest standard deviation, 4.55 hours, which means it has a much fairer assignment process than both RL and heuristic models (5.43 and 6.36 hours, respectively). This shows that the RL and heuristic models have a greater tendency toward disparity in faculty workload. This reflects the nuanced judgment of human schedulers, weighing institutional needs against faculty equity in ways automated processes currently cannot match.

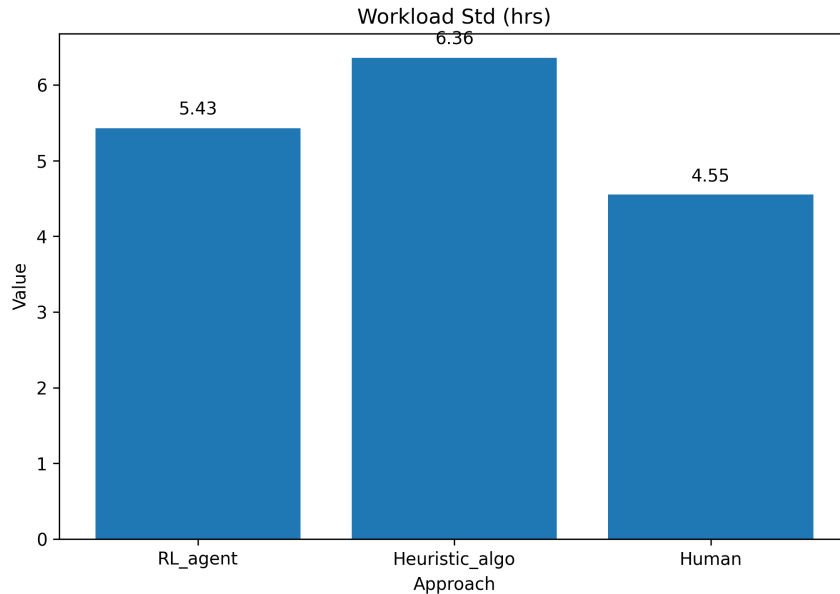


Figure 4: Standard deviation of assigned faculty workloads (hours) for each scheduling method. Lower values indicate greater equity.

Schedule Compactness Figure 5 reveals variations in the compactness of the schedule, operationalized by the average in-day gap between assignments. The heuristic algorithm creates the most compact schedules, 85.8 minutes, which might help improve efficiency and reduce slack. Manual scheduling is moderately compact, with an 88.5 minute average, thus possibly balancing efficiency with realistic feasibility. In contrast, the RL agent generates the least compact schedules, at 95.9 minutes, with longer gaps between assignments being common. This trend indicates that, while automated systems may excel at optimizing a particular dimension schedule compactness in this case these gains can come at the expense of other important scheduling objectives.

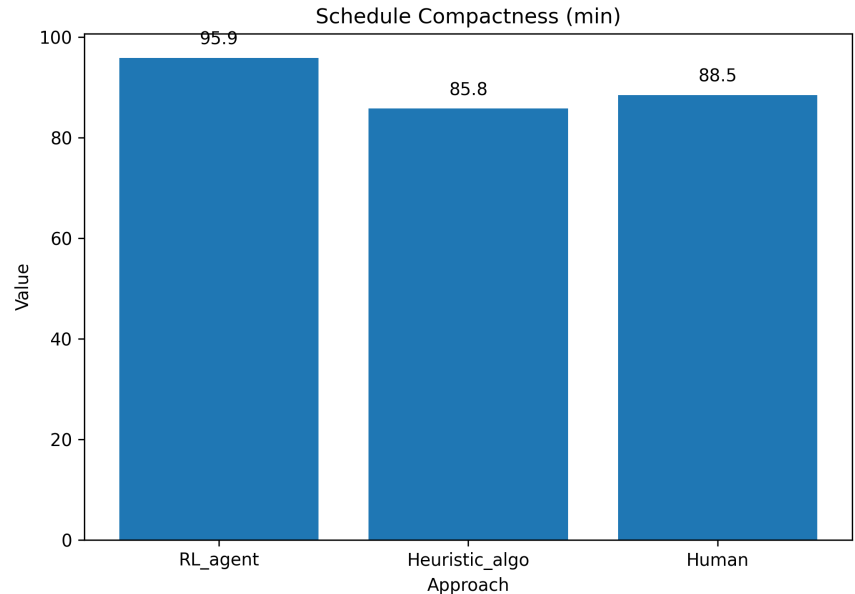


Figure 5: Average schedule compactness (gap between assignments, in minutes) for each method. Lower values reflect more compact schedules.

Feasibility and Coverage The completeness of scheduling—measured as the number of unassigned course sections—is visualized in Figure 6. Human scheduling consistently achieves total coverage, with all courses assigned. In stark contrast, the heuristic and RL-based methods leave 28 and 118 course sections unassigned, respectively, indicating significant shortfalls in automated scheduling feasibility. This gap points to a critical challenge for automation: balancing optimal assignments with the overarching requirement of full course coverage.

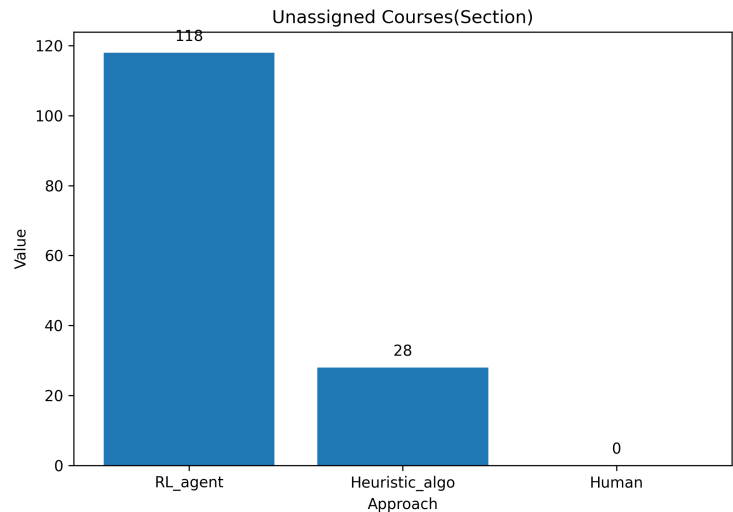


Figure 6: Number of unassigned course sections for each scheduling method.

Conflict Avoidance Table 4 reveals the prevalence of time conflicts under each approach. Both manual and RL-based schedules successfully avoid all conflicts, showcasing their effectiveness at upholding this crucial institutional requirement. The heuristic method, however, introduces 49 time conflicts, exposing a vulnerability in rule-based strategies, particularly when constraints interact in complex ways. This outcome highlights the importance of robust constraint-checking mechanisms, especially in automated systems.

Workload Inequality Workload inequality, as quantified by the Gini coefficient (Figure 7), further differentiates the fairness of each approach. The manual method achieves the lowest Gini coefficient (0.155), demonstrating superior

Table 4: Number of Time Conflicts for Each Approach

Approach	Time Conflicts
Human	0
RL Agent	0
Heuristic	49

equity in workload assignment. The RL and heuristic strategies, with coefficients of 0.211 and 0.232, respectively, display greater disparities. This finding reinforces the conclusion that human schedulers, despite being slower and potentially less scalable, can better internalize and balance institutional priorities with fairness.

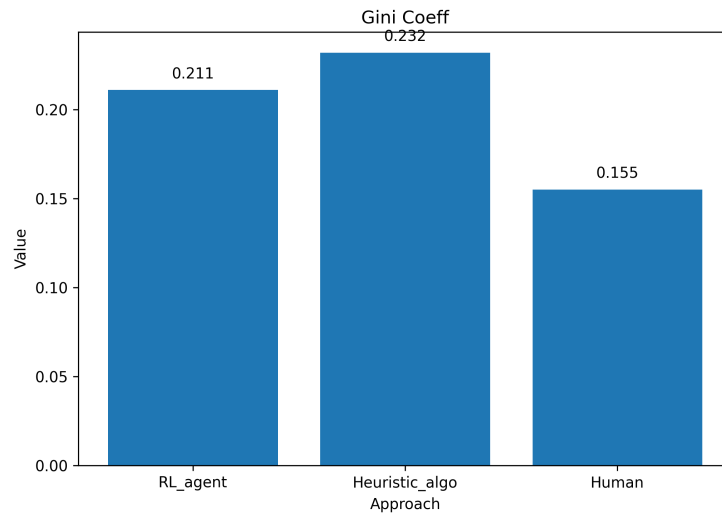


Figure 7: Gini coefficient for faculty workload distribution under RL agent, heuristic algorithm, and human scheduling. Lower values indicate greater equity.

Faculty Preference Satisfaction As shown in Figure 8, faculty preference satisfaction is maximized by the RL agent (100%), closely followed by the heuristic method (99.36%), while manual scheduling lags behind (77.29%). This suggests that, when optimized directly for stated preferences, automated systems can achieve near-perfect satisfaction rates. However, these improvements must be carefully weighed against declines in coverage, fairness, and conflict avoidance.

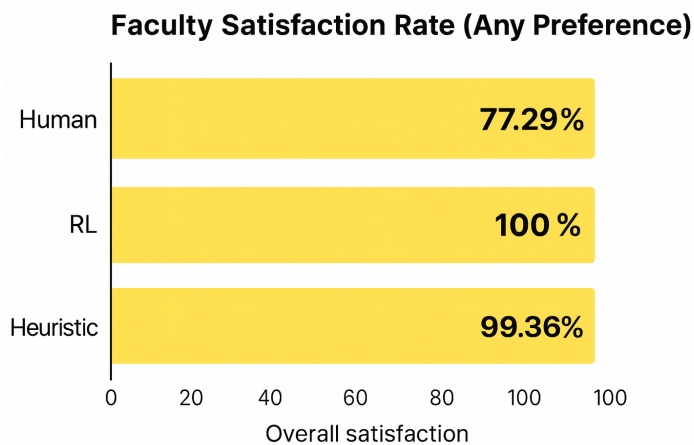


Figure 8: Overall faculty preference satisfaction rates under human, RL, and heuristic scheduling approaches.

4.3 Analysis of the "Unassigned Courses" Issue

One of the most striking weaknesses of the RL-based scheduling approach was that it could not attain full coverage of courses, leaving 118 sections unassigned compared to 28 for the heuristic algorithm and zero for the human-generated schedule. This gap likely resulted from both structural and algorithmic issues inherent in the current design. First, the reward function is designed in a way that encourages the agent to secure a small number of "high reward" assignments those which strongly align with faculty preference or have few scheduling conflicts rather than to consider suboptimal yet necessary matches for harder to assign courses. Because preference satisfaction and conflict avoidance yield immediately positive feedback, the agent is liable to converge prematurely on locally optimal actions that maximize those components at the expense of overall coverage.

Another related factor that could cause this behavior is the duration of training and sparsity of the reward signal. For a large combinatorial action space, full coverage will be a delayed reward encountered only after many valid assignments have been made. The training horizon of 500,000 timesteps may be too short to explore such long-horizon dependencies during policy training. Extending the training further or employing a curriculum learning, where the agent learns simpler versions of the scheduling task first, would likely improve sample efficiency.

Finally, the lack of explicit action masking or mechanisms of constrained exploration might have lowered training efficiency. If an agent is often sampling invalid or infeasible assignments (for instance, faculty already at capacity or not qualified to teach a course), learning slows down a lot because of the continuous stream of negative rewards. On the other hand, masking rules that are too strict may prematurely limit exploration and prevent an agent from finding complex, multistep strategies that would enable better coverage. For this reason, future variants should also investigate adaptive or soft masking methods that dynamically balance exploration of hard assignments with respect for institutional constraints. Collectively, these factors indicate that the RL agent's unassigned courses are not due to an inherent limitation in reinforcement learning but rather from the interaction of several elements: reward design, training time, and action space management. A focus on improving such areas in future versions could lead to substantially improved model performance in formulating complete and well balanced schedules comparable to those generated by humans.

4.4 Department-Specific Evaluation: Mathematics Department Case Study

To further assess the generalization capability of the proposed reinforcement learning model, an additional experiment was carried out using real scheduling data from the Mathematics Department. Unlike the Computer Science dataset, which includes both theory and laboratory courses and exhibits high variability in faculty eligibility constraints, the Mathematics dataset consisted of only six theory-based courses and a relatively small, homogeneous faculty group. All other experimental parameters such as the reward function, PPO configuration, and environment structure—were kept identical to maintain consistency.

Under these simplified conditions, the RL agent demonstrated notably improved performance, successfully assigning almost all courses and leaving only 10 sections unallocated. This result indicates that the model performs with greater stability and coverage in more uniform environments where constraint diversity is limited. The absence of laboratory components and the presence of simpler eligibility mappings effectively reduced the complexity of the action space, enabling faster convergence and more consistent policy behavior.

These results imply that the shortcomings observed in the Computer Science Department experiment particularly the larger number of unassigned courses stem primarily from the heterogeneity and complexity of the dataset rather than any intrinsic limitations of the RL framework itself. Consequently, a department level deployment approach could represent a viable path toward gradual institutional scaling, wherein the agent is initially applied to smaller, domain specific scheduling tasks before being extended to multi department or mixed course environments.

4.5 Interpretation and Discussion

The comparative results presented above underscore the multifaceted nature of the scheduling problem, and the limitations inherent in both manual and automated approaches. Automated algorithms whether RL or heuristic can be finely tuned to optimize select metrics, such as preference satisfaction or schedule compactness. However, this often introduces trade offs, resulting in increased workload disparity, partial coverage, or higher susceptibility to conflicts.

Manual scheduling, while less efficient in maximizing explicit preference satisfaction, excels in holistic objectives: achieving full coverage, fairness, and robust avoidance of conflicts. This highlights the implicit value of human judgment, adaptability, and domain expertise in synthesizing multiple, sometimes competing, constraints.

Summary of Key Findings:

- **Manual (Human) Scheduling:** Consistently achieves the highest levels of feasibility (complete coverage), equity (lowest workload standard deviation and Gini coefficient), and conflict avoidance, though with only moderate faculty satisfaction.
- **RL-Based Scheduling:** Excels at optimizing explicit preference satisfaction and avoiding conflicts, but suffers from high workload disparity, low schedule compactness, and poor coverage (large number of unassigned courses).
- **Heuristic Scheduling:** Produces the most compact schedules and achieves near-perfect preference satisfaction, yet manifests significant workload imbalances, high idle times, frequent time conflicts, and incomplete coverage.

These results collectively indicate that, while automated scheduling solutions show great promise for optimization and scalability, their real-world deployment must be carefully designed to address the full spectrum of institutional needs. Future research should focus on developing hybrid approaches that integrate human judgment with machine-driven optimization, thereby achieving both automation and fairness.

5 Conclusion

This work provides new insights into the operational challenges and trade offs in faculty course allocation through a comparative analysis of manual, RL, and heuristic-based scheduling strategies. The empirical findings show that even with the advancement of algorithmic scheduling, human-devised solutions continue to excel in providing balanced workloads, full course coverage, and conflict free timetables. These results reflect nuances in decision making by experienced schedulers for complex institutional environments.

Automated methods, particularly RL and heuristic-driven, showed excellent performance in both optimizing faculty preference satisfaction and schedule compactness. This often came at the cost of increased workload disparity, higher idle times, and, for heuristic methods, increased scheduling conflicts and incomplete course assignments. The tendency for algorithmic solutions to optimize singular objectives at the expense of broader institutional requirements was a key limitation.

These all point to the need for integrated scheduling frameworks that balance automation with institutional fairness and adaptability. The study indicates that hybrid methodologies can offer the most robust path forward by leveraging computational efficiency along with expert oversight. Further research is needed in developing multi-objective optimization techniques and more sophisticated constraint management in order to bridge the gap between algorithmic efficiency and comprehensive performance presently achieved by human schedulers.

5.1 Discussion of Scalability and Real-World Deployment

While the proposed RL approach shows significant promise in optimizing specific metrics of preference satisfaction, its real world deployment depends on how well major challenges like scalability and practical integration are solved. Our experiments, conducted on a single semester dataset, have highlighted a number of considerations for scaling this system to a larger university context or making it responsive to dynamic academic environments.

Computational and Practical Scaling Challenges: Among these, the computation cost of training is a major concern. The current model was trained over 500,000 timesteps on a fixed set of courses and faculty. In a larger university, the action space, the product of the number of faculty and courses, would grow quadratically. It is likely this would result in a need for a corresponding increase in training time and resources to achieve convergence, which could make frequent retraining prohibitive. Second, the state representation, comprised of a preference and assignment vector for every faculty member, would be increasingly highdimensional, requiring more complex policy networks and risking the curse of dimensionality. Future work should consider more efficient state representations such as graph neural networks to model relational data and transfer learning with the goal of leveraging policies trained on smaller departments or previous semesters.

Another important consideration is dynamic adaptability. In real world academic environments, very few schedules are truly static: a faculty member may become unavailable, new courses can be introduced, or fluctuations in enrollment may require adjusting sections. To accommodate these dynamic changes, future versions of the system could utilize continual learning or incremental policy updates that allow the RL agent to update previously learned policies without retraining from scratch. Integrating modules for change detection and adaptive re-optimization would further enhance the robustness of the system during mid semester disruptions. From an operational perspective, a hybrid deployment model is the most promising. In this paradigm, the RL agent could generate an initial “draft” schedule optimized for constraints and preferences, which human administrators then refine and finalize based on contextual or policy specific considerations. Such a collaborative framework combines scalability and speed through RL driven automation with

the interpretability and institutional insight of human schedulers. Over time, feedback from human adjustments might also feed into the RL environment to improve policy learning, leading to a progressively smarter and more reliable scheduling assistant. However, real scalability will require not only computational efficiency but also the integration of institutional data pipelines, user interfaces, and feedback loops that can support continuous learning. Future research should focus on distributed RL training, hybrid decision workflows, and adaptive scheduling policies that maintain performance as the scheduling environment evolves semester over semester.

A Appendix : Faculty Workload and Idle-time Distributions

Table A1: Workload Summary Table (Manual Scheduling)

Faculty Name	Idle Time (hrs)
KAWSER IROM RUSHEE	28.00
SAMIHA ISLAM ABRITA	28.00
MD. ARIFUL ISLAM	25.00
AKIB AHMED	24.00
MD. MUSTAK UN NOBI	23.33
Instructor-2	2.33
Instructor-1	2.33
Instructor-3	2.33
Instructor-9	2.33
MEHEDI SAYED	2.33

Table A2: Workload Statistics (Manual Scheduling)

Metric	Value (hrs)
Max Workload	20.33
Min Workload	6.00
Workload Range	14.33
Average Idle Time	14.28

Table A3: Workload Statistics for RL Scheduling

Metric	Value (hrs)
Max Workload	20.67
Min Workload	3.00
Workload Range	17.67
Average Idle Time per Faculty	48.36

Table A4: Faculty Idle Time for RL Scheduling

Faculty Name	Idle Time (hrs)
SULTANUL ARIFEEN HAMIM	89.33
KAZI SADIA	86.67
MIR MD. KAWSUR	86.67
SAEEDA SHARMEEN RAHMAN	86.67
RIFAT AL MAMUN RUDRO	86.00
DR. KHANDAKER TABIN HASAN	3.33
DR. MD. ABDULLAH-AL-JUBAIR	3.33
DR. MD IFTEKHARUL MOBIN	3.33
JUENA AHMED NOSHIN	3.33
SIRAJUM MUNIRA SHIFAT	2.33

Table A5: Workload Statistics for Heuristic Scheduling

Metric	Value (hrs)
Max Workload	26.00
Min Workload	4.33
Workload Range	21.67
Average Idle Time per Faculty	41.26

Table A6: Faculty Idle Time for Heuristic Scheduling

Faculty Name	Idle Time (hrs)
AIMAN LAMEESA	104.33
DR. MD. MANZURUL HASAN	90.00
NUSRAT JAHAN ANANNYA	86.67
MD. MAZID-UL-HAQUE	86.33
DR. MOHAMMAD RABIUL ISLAM	86.00
DR. MD. ABDULLAH-AL-JUBAIR	3.33
PROF. DR. DIP NANDI	3.33
INSTRUCTOR-4	2.33
SYED NAFIUL SHEFAT	2.00
SAIKAT BAUL	0.00

References

- [1] R Lewis. A survey of metaheuristic-based techniques for university timetabling problems. *OR Spectrum*, 30(1):167–190, 2008.
- [2] Edmund K. Burke, John P. Newall, and R. F. Weare. A hybrid genetic algorithm for highly constrained exam timetabling problems. *Applied Artificial Intelligence*, 16(4):337–356, 2002.
- [3] Patrick Kenekayoro. Incorporating machine learning to evaluate solutions to the university course timetabling problem. *International Journal of Computer Science and Network Security*, 19(10):22–31, 2019.
- [4] K. Umamaheswari, S. S. Bhushan, et al. Revolutionizing departmental management through intelligent technology: A machine learning approach to optimization. In *Proceedings of the IEEE Conference on Intelligent Systems*. IEEE, 2025.
- [5] Sami Ahmed Haider, Khwaja Mutahir Ahmad, Adnan Zahid, Azzah AlGhamdi, Ismail Keshta, and Mukesh Soni. Genetic and deep reinforcement learning-based intelligent course scheduling for smart education. In *2024 the International Conference on Artificial Intelligence and Teacher Education (ICAITE)*, pages 117–124. ACM, 2024.
- [6] A Ghaffar, IU Din, A Tariq, and MH Zafar. Hybridization and artificial intelligence in optimizing university examination timetabling problem: A systematic review. *Review of Education*, 2025.
- [7] Chien-Liang Liu, Chuan-Chin Chang, and Chun-Jan Tseng. Actor-critic deep reinforcement learning for solving job shop scheduling problems. *IEEE Access*, 8:71752–71762, 2020.
- [8] Giovanni Bonetta, Davide Zago, Rossella Cancelliere, and Andrea Grosso. Job shop scheduling via deep reinforcement learning: a sequence to sequence approach, 2023.
- [9] Wen Song, Xinyang Chen, Qiqiang Li, and Zhiguang Cao. Flexible job-shop scheduling via graph neural network and deep reinforcement learning. *IEEE Transactions on Industrial Informatics*, 19(2):1600–1610, 2023.
- [10] Junyoung Park, Jaehyeong Chun, Sang Hun Kim, Youngkook Kim, and Jinkyoo Park. Learning to schedule job-shop problems: Representation and policy learning using graph neural network and reinforcement learning. *International Journal of Production Research*, 59(11):3360–3377, 2021.
- [11] Mohammed Sharafath Abdul Hameed and Andreas Schwung. Graph neural networks-based scheduler for production planning problems using reinforcement learning. *Journal of Manufacturing Systems*, 69:91–102, 2023.
- [12] Yu Pu, Fang Li, and Shahin Rahimifard. Multi-agent reinforcement learning for job shop scheduling in dynamic environments. *Sustainability*, 16(8):3234, 2024.
- [13] Haixu Yu and Xudong Zhao. Deep reinforcement learning with reward design for quantum control. *IEEE Transactions on Artificial Intelligence*, 5(3):1087–1101, 2024.
- [14] Steffen Gracla, Edgar Beck, Carsten Bockelmann, and Armin Dekorsy. Robust deep reinforcement learning scheduling via weight anchoring. *arXiv preprint arXiv:2304.10176*, 2023.

- [15] Yiwei Guo. A reinforcement learning approach to train timetabling for inter-city high speed railway lines. In *2020 IEEE 5th International Conference on Intelligent Transportation Engineering*, pages 493–497. IEEE, 2020.
- [16] Maziyar Khadivi, Todd Charter, Marjan Yaghoubi, Masoud Jalayer, Maryam Ahang, Ardeshtir Shojaeinasab, and Homayoun Najjaran. Deep reinforcement learning for machine scheduling: Methodology, the state-of-the-art, and future directions. *SSRN Electronic Journal*, 2023.
- [17] Nhat T. Nguyen, T. Matsuo, F. L. Gaol, and Y. Manolopoulos. Applications of deep reinforcement learning in intelligent education systems. In *Proceedings of the 17th Asian Conference on Intelligent Information and Database Systems (ACIIDS)*. Springer, 2025.
- [18] Yuping Fan, Zhiling Lan, Taylor Childers, Paul Rich, William Allcock, and Michael E Papka. Deep reinforcement agent for scheduling in hpc. In *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 807–816, 2021.
- [19] Edmund K Burke, John P Newall, and R F Weare. A hybrid genetic algorithm for highly constrained exam timetabling problems. *Applied Artificial Intelligence*, 16(4):337–356, 2002.
- [20] K Umamaheswari, SS Bhushan, et al. Revolutionizing departmental management through intelligent technology: A machine learning approach to optimization. In *Proceedings of the IEEE Conference on Intelligent Systems*. IEEE, 2025.
- [21] Jonathan Bassen, Bharathan Balaji, Michael Schaarschmidt, Jay Painter, Dawn Zimmaro, Alex Games, Ethan Fast, Candace Thille, and John Mitchell. Reinforcement learning for the adaptive scheduling of educational activities. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, 2020.
- [22] Jie Xu, Tianwei Xing, and Mihaela van der Schaar. Personalized course sequence recommendations. *IEEE Transactions on Signal Processing*, 64(23):6246–6259, 2016.
- [23] Yu Pu, Fang Li, and Shahin Rahimifard. Multi-agent reinforcement learning for job shop scheduling in dynamic environments. *Sustainability*, 16(8):3234, 2024.
- [24] Ciprian Paduraru, Catalina Camelia Patilea, and Stefan Iordache. Task scheduling: A reinforcement learning based approach. In *Proceedings of the 15th International Conference on Agents and Artificial Intelligence (ICAART 2023)*, pages 948–955, 2023.
- [25] Sushil Kumar Jain and Jyoti. HDRLGA: Hybrid deep reinforcement learning and genetic algorithm task scheduling approach in cloud computing. *Library Progress International*, 44(3):15815–15822, 2024.
- [26] Burak Demirel, Arunselvan Ramaswamy, Daniel E Quevedo, and Holger Karl. DEEPCAS: A deep reinforcement learning algorithm for control-aware scheduling. *IEEE Control Systems Letters*, 2(4):737–742, 2018.
- [27] Russell Goh, NR Sabar, and Graham Kendall. Enhanced simulated annealing for course timetabling with adaptive reheating and learned neighborhoods. *Computers & Operations Research*, 100:85–98, 2018.
- [28] Nhat T Nguyen, T Matsuo, FL Gaol, and Y Manolopoulos. Applications of deep reinforcement learning in intelligent education systems. In *Proceedings of the 17th Asian Conference on Intelligent Information and Database Systems (ACIIDS)*. Springer, 2025.